

iOS面试题合集（RunLoop篇）

iOSer iOSer 5月19日

收录于话题

#BAT大厂面试题合集

9个

欢迎点击上方蓝字“iOSer”关注我们
再点击右上角“...”菜单，选择“设为星标”
我们一起精进、成长！

本栏目将持续更新--请iOS的小伙伴关注我们!

做这个的初心是希望能巩固自己的基础知识，

当然也希望能帮助更多的开发者，

可以加入我们的技术交流群：**834688868**，群里每天都会有干货、技术动向、职业生涯、行业热点、职场趣事等一切有关于程序员的内容分享。

不管你是大牛还是小白都欢迎入驻，分享BAT，阿里面试题、面试经验，讨论技术，大家一起交流学习成长！

目录：

- 1.Runloop 和线程的关系？
- 2.RunLoop的运行模式
- 3.runloop内部逻辑？
- 4.autoreleasePool 在何时被释放？
5. GCD 在RunLoop中的使用？
- 6.AFNetworking 中如何运用 Runloop？
7. PerformSelector 的实现原理？
- 8.PerformSelector:afterDelay:这个方法在子线程中是否起作用？
- 9.事件响应的过程？
- 10.手势识别的过程？

11.CADisplayTimer和Timer哪个更精确？

1.Runloop 和线程的关系？

- 一个线程对应一个 Runloop。
- 主线程的默认就有了 Runloop。
- 子线程的 Runloop 以懒加载的形式创建。
- Runloop 存储在一个全局的可变字典里，线程是 key，Runloop 是 value。

2.RunLoop的运行模式有哪些？

RunLoop的运行模式共有5种，RunLoop只会运行在一个模式下，要切换模式，就要暂停当前模式，重写启动一个运行模式



- kCFRunLoopDefaultMode，App的默认运行模式，通常主线程是在这个运行模式下运行
- UITrackingRunLoopMode，跟踪用户交互事件（用于 ScrollView 追踪触摸滑动，保证界面滑动时不受其他Mode影响）
- kCFRunLoopCommonModes，伪模式，不是一种真正的运行模式
- UIInitializationRunLoopMode：在刚启动App时第进入的第一个Mode，启动完成后就不再使用
- GSEventReceiveRunLoopMode：接受系统内部事件，通常用不到

3.runloop内部逻辑？

- 实际上 RunLoop 就是这样一个函数，其内部是一个 do-while 循环。当你调用 CFRunLoopRun() 时，线程就会一直停留在这个循环里；直到超时或被手动停止，该函

数才会返回。

- 内部逻辑：
 - 如果是 Timer 事件，处理 Timer 并重新启动循环，跳到第 2 步
 - 如果输入源被触发，处理该事件（文档上是 deliver the event）
 - 如果 RunLoop 被手动唤醒但尚未超时，重新启动循环，跳到第 2 步
 - 事件到达基于端口的输入源（port-based input sources）（也就是 Source0）
 - Timer 到时间执行
 - 外部手动唤醒
 - 为 RunLoop 设定的时间超时

1. 通知 Observer 已经进入了 RunLoop
2. 通知 Observer 即将处理 Timer
3. 通知 Observer 即将处理非基于端口的输入源（即将处理 Source0）
4. 处理那些准备好的非基于端口的输入源（处理 Source0）
5. 如果基于端口的输入源准备就绪并等待处理，请立刻处理该事件。转到第 9 步（处理 Source1）
6. 通知 Observer 线程即将休眠
7. 将线程置于休眠状态，直到发生以下事件之一
8. 通知 Observer 线程刚被唤醒（还没处理事件）
9. 处理待处理事件

4.autoreleasePool 在何时被释放？

- App启动后，苹果在主线程 RunLoop 里注册了两个 Observer，其回调都是 `_wrapRunLoopWithAutoreleasePoolHandler()`。
- 第一个 Observer 监视的事件是 Entry(即将进入Loop)，其回调内会调用 `_objc_autoreleasePoolPush()` 创建自动释放池。其 order 是 -2147483647，优先级最高，保证创建释放池发生在其他所有回调之前。
- 第二个 Observer 监视了两个事件：BeforeWaiting(准备进入休眠) 时调用 `_objc_autoreleasePoolPop()` 和 `_objc_autoreleasePoolPush()` 释放旧的池并创建新池；Exit(即将退出Loop) 时调用 `_objc_autoreleasePoolPop()` 来释放自动释放池。这个 Observer 的 order 是 2147483647，优先级最低，保证其释放池子发生在其他所有回调

之后。

- 在主线程执行的代码，通常是写在诸如事件回调、Timer回调内的。这些回调会被 RunLoop 创建好的 autoreleasePool 环绕着，所以不会出现内存泄漏，开发者也不必显示创建 Pool 了。

5.GCD 在RunLoop中的使用？

- GCD由 子线程 返回到 主线程,只有在这种情况下才会触发 RunLoop。会触发 RunLoop 的 Source 1 事件。

6.AFNetworking 中如何运用 Runloop？

AFURLConnectionOperation 这个类是基于 NSURLConnection 构建的，其希望能在后台线程接收 Delegate 回调。

为此 AFNetworking 单独创建了一个线程，并在这个线程中启动了一个 RunLoop：



```
+ (void)networkRequestThreadEntryPoint:(id)__unused object {
    @autoreleasepool {
        [[NSThread currentThread] setName:@"AFNetworking"];
        NSRunLoop *runLoop = [NSRunLoop currentRunLoop];
        [runLoop addPort:[NSMachPort port] forMode:NSDefaultRunLoopMode];
        [runLoop run];
    }
}

+ (NSThread *)networkRequestThread {
    static NSThread *_networkRequestThread = nil;
    static dispatch_once_t oncePredicate;
```

```
dispatch_once(&oncePredicate, ^{
    _networkRequestThread = [[NSThread alloc] initWithTarget:self selector:@selector(networkRequestThreadEntryPoint:) object:nil];
    [_networkRequestThread start];
});
return _networkRequestThread;
}
```

RunLoop 启动前内部必须要有至少一个 Timer/Observer/Source，所以 AFNetworking 在 [runLoop run] 之前先创建了一个新的 NSMachPort 添加进去了。

通常情况下，调用者需要持有这个 NSMachPort (mach_port) 并在外部线程通过这个 port 发送消息到 loop 内；但此处添加 port 只是为了让 RunLoop 不至于退出，并没有用于实际的发送消息。



```
- (void)start {
    [self.lock lock];
    if ([self isCancelled]) {
        [self performSelector:@selector(cancelConnection) onThread:[self class] networkRequestThread withObject:nil waitUntilDone:NO modes:[self.runLoopModes allObjects]];
    } else if ([self isReady]) {
        self.state = AFOperationExecutingState;
        [self performSelector:@selector(operationDidStart) onThread:[self class] networkRequestThread withObject:nil waitUntilDone:NO modes:[self.runLoopModes allObjects]];
    }
    [self.lock unlock];
}
```

当需要这个后台线程执行任务时，AFNetworking 通过调用 [NSObject performSelector:onThread:...] 将这个任务扔到了后台线程的 RunLoop 中。

7.PerformSelector 的实现原理?

- 当调用 NSObject 的 performSelector:afterDelay: 后，实际上其内部会创建一个 Timer 并添加到当前线程的 RunLoop 中。所以如果当前线程没有 RunLoop，则这个方法会失效。
- 当调用 performSelector:onThread: 时，实际上其会创建一个 Timer 加到对应的线程去，同样的，如果对应线程没有 RunLoop 该方法也会失效。

8.PerformSelector:afterDelay:这个方法在子线程中是否起作用?

- 不起作用，子线程默认没有 Runloop，也就没有 Timer。可以使用 GCD的 dispatch_after来实现

9.事件响应的过程?

- 苹果注册了一个 Source1 (基于 mach port 的) 用来接收系统事件，其回调函数为 __IOHIDEventSystemClientQueueCallback()。
- 当一个硬件事件(触摸/锁屏/摇晃等)发生后，首先由 IOKit.framework 生成一个 IOHIDEvent 事件并由 SpringBoard 接收。这个过程的具体情况可以参考这里。SpringBoard 只接收按键(锁屏/静音等)，触摸，加速，接近传感器等几种 Event，随后用 mach port 转发给需要的 App 进程。随后苹果注册的那个 Source1 就会触发回调，并调用 _UIApplicationHandleEventQueue() 进行应用内部的分发。
- _UIApplicationHandleEventQueue() 会把 IOHIDEvent 处理并包装成 UIEvent 进行处理或分发，其中包括识别 UIGesture/处理屏幕旋转/发送给 UIWindow 等。通常事件比如 UIButton 点击、touchesBegin/Move/End/Cancel 事件都是在这个回调中完成的。

10.手势识别的过程?

- 当 `_UIApplicationHandleEventQueue()` 识别了一个手势时，其首先会调用 `Cancel` 将当前的 `touchesBegin/Move/End` 系列回调打断。随后系统将对应的 `UIGestureRecognizer` 标记为待处理。
- 苹果注册了一个 `Observer` 监测 `BeforeWaiting` (Loop即将进入休眠) 事件，这个 `Observer` 的回调函数是 `_UIGestureRecognizerUpdateObserver()`，其内部会获取所有刚被标记为待处理的 `GestureRecognizer`，并执行 `GestureRecognizer` 的回调。
- 当有 `UIGestureRecognizer` 的变化(创建/销毁/状态改变)时，这个回调都会进行相应处理。

11.CADisplayTimer和Timer哪个更精确

CADisplayLink 更精确

- iOS设备的屏幕刷新频率是固定的，`CADisplayLink`在正常情况下会在每次刷新结束都被调用，精确度相当高。
- `NSTimer`的精确度就显得低了点，比如`NSTimer`的触发时间到的时候，`RunLoop`如果在阻塞状态，触发时间就会推迟到下一个`RunLoop`周期。并且 `NSTimer`新增了`tolerance`属性，让用户可以设置可以容忍的触发的时间的延迟范围。
- `CADisplayLink`使用场合相对专一，适合做UI的不停重绘，比如自定义动画引擎或者视频播放的渲染。`NSTimer`的使用范围要广泛的多，各种需要单次或者循环定时处理的任务都可以使用。
- 在UI相关的动画或者显示内容使用 `CADisplayLink`比起用`NSTimer`的好处就是我们不需

要在格外关心屏幕的刷新频率了，因为它本身就是跟屏幕刷新同步的。

部分总结的RunLoop面试题就到这里了，
以上内容对你有帮助的话，记得关注我们！

文章来源于网络，已尽可能标明作者以及来源，文章内容为作者独立观点，不代表本公众号立场，因文章侵权本公众号不承担任何法律及连带责任，如有侵权，请联系我们删除。



觉得不错的話，别忘了点个"在看"哦~ 📌

收录于话题 #BAT大厂面试题合集·9个

上一篇

[iOS面试题合集（架构+设计模式篇）](#)

下一篇

[iOS面试题合集（Runtime篇）](#)

[阅读原文](#)

喜欢此内容的人还喜欢

iOS技术人员如何做晋升答辩？

iOSer

退圈的香港明星，到底有多穷？

蝉创意

当紫薇成为主播，当年让我流泪的她跑哪儿去了

新闻哥